

Code Review Agent – Developer Guide

This guide explains how to use the `/code_review` agent within the MiDAS GitHub workflow to conduct structured and comprehensive code reviews.

Introduction

The `/code_review` agent is an expert code reviewer integrated into the MiDAS GitHub workflow. It is triggered through GitHub issue or pull request comments and performs systematic, multi-dimensional code evaluations without modifying files.

The agent reviews code across quality, functionality, security, performance, testing, and best-practice dimensions. It organizes feedback by severity level and provides actionable recommendations with clear explanations.

The purpose of the `/code_review` agent is to improve code quality, reduce defects, and standardize review practices across the organization.

What Are the Use Cases for the Code Review Agent

The `/code_review` agent should be used when a structured and objective review of code changes is required.

Common Use Cases

- Reviewing a pull request before merge
- Evaluating a newly implemented feature
- Reviewing refactored modules
- Assessing security vulnerabilities
- Identifying performance issues
- Improving readability and maintainability



When to Use `/code_review`

Use `/code_review` when:

- Code has been implemented and pushed.
- A pull request is open for review.
- You want structured and severity-based feedback.
- You want validation across security, performance, and testing dimensions.

Do not use `/code_review` for planning or architectural design discussions. Use `/design` or `/plan` for those scenarios .

How to Use the Code Review Agent

The `/code_review` agent is triggered from GitHub comments.

Step 1: Create a pull request

The agent works best when:

- A pull request is open, or
- The issue includes code changes or file references.

Ensure the changes are visible in the GitHub diff view.



Ask



Step 2: Trigger the Agent

In the issue or pull request comment section, type:

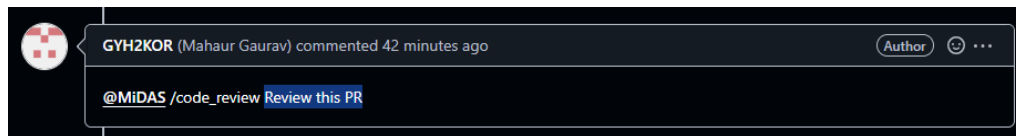
```
@MiDAS /code_review <optional context>
```

Example:

```
@MiDAS /code_review Review this implementation for quality, security, and performance.
```

Another example:

```
@MiDAS /code_review Focus on input validation and error handling in this PR.
```



Step 3: Review the Generated Feedback

The agent will respond with structured feedback organized by severity levels.

The output may include:

- Critical Issues
- Major Concerns
- Suggestions
- Minor Notes

It may also include:

- File references
- Code line observations
- Security vulnerability analysis
- Performance considerations
- Test coverage gaps
- Best-practice recommendations



github-actions

bot

commented 37 minutes ago • edited

MiDAS finished @GYH2KOR's task in 1m 45s — [View job](#)

Code Review Summary

- ✓ Read repository guidelines (CLAUDE.md)
- ✓ Review changed files
- ✓ Analyze code quality and best practices
- ✓ Check test coverage
- ✓ Provide comprehensive feedback

Review Feedback

I've reviewed the PR that adds a health check endpoint to SS6 service. Overall, the implementation is solid and follows the existing patterns in the codebase. Here's my detailed feedback:

Strengths

- Consistent with existing patterns:** The implementation follows the same structure as other health check endpoints across the platform (SS1, SS2, SS3, etc.)
- Proper integration:** The router is correctly registered in `appmain.py:175`
- Good test coverage:** Two unit tests cover basic functionality and response structure validation
- Documentation updated:** The API endpoint mapping in `docs/api_endpoints.md:92` is properly updated

Issues and Recommendations

- Missing Response Model (Optional Enhancement)**

Severity: Low

Some subsystems (like SS5-orchestration) use a response model for health checks. Consider adding one for consistency:

File: `ss6-serviceapi-platform/app/api/v1/health_management/health_router.py:7`

Current:

```
@router.get("/services/api/v1/health")
async def health_check():
```

Consider:



```
@router.get("/services/api/v1/health", tags=["Health"])
async def health_check():
```

Adding the `tags` parameter helps organize the API documentation in Redoc/Swagger UI. See SS3's implementation at `ss3-contextcatalogue/app/api/v1/health_management/health_router.py:8`.

2. Response Format Inconsistency

Severity: Medium

The health check returns a bare JSON object `{"status": "..."}` , but according to the repository's API documentation (`docs/api_endpoints.md:168`):

└ All API responses: The actual response payload is always contained within the `data` field of the JSON response.

However, I notice that SS1, SS2, and other health endpoints also return bare objects. This appears to be an accepted pattern for health checks, but it's worth clarifying:

- If health checks are exempt from the `data` wrapper pattern: Document this exception
- If they should follow the pattern: Consider wrapping the response in a `data` field for consistency

3. Test Path Setup Complexity

Severity: Low

File: `ss6-serviceapi-platform/tests/unit/api/test_health_router.py:6-15`

The path setup logic is more complex than necessary:

```
path = Path(__file__)
app_path = path.parts[
    : path.parts.index("ss6-serviceapi-platform") + 1
] # Get the path till ss6-serviceapi-platform
app_path += ("app/",)
app_root_pth = Path(*app_path)
sys.path.append(str(app_root_pth))
parent_path = app_root_pth.resolve().parent.parent
sys.path.append(str(parent_path))
```

This is consistent with other SS6 tests (see `test_common_utils.py:12-19`), so it follows the established pattern. No change needed, but consider simplifying across the entire test suite in a future refactor.

4. Response Middleware Interaction

Severity: Low - Verification Needed

The health endpoint will be wrapped by `ResponseMiddleware` (`appmain.py:122`). Verify that:



```
@router.get("/services/api/v1/health", status_code=status.HTTP_200_OK, tags=["Health"])
```

See SS3's implementation at `ss3-contextcatalogue/app/api/v1/health_management/health_router.py:7-8`.

Code Quality

- ✓ Proper imports and logging
- ✓ Clear docstring with examples
- ✓ Follows async pattern
- ✓ No security vulnerabilities
- ✓ No performance concerns

Test Coverage

The tests are well-structured and cover:

- ✓ Basic endpoint functionality
- ✓ Status code verification
- ✓ Response structure validation
- ✓ String type checking

Minor suggestion: Consider adding a test to verify the endpoint works through the full application with middleware applied:

```
def test_health_check_through_app():
    """Test health endpoint with full application context including middleware."""
    from app.appmain import app
    client = TestClient(app)
    response = client.get("/services/api/v1/health")
    assert response.status_code == 200
```

Documentation

The API endpoint mapping table is correctly updated at line 92. The entry is well-formatted and consistent with other subsystems.

Summary

This is a clean, well-implemented feature that adds essential health check functionality to SS6. The code follows existing patterns and includes appropriate tests. The suggestions above are mostly optional enhancements for consistency and clarity.

Recommendation: ✓ Approve (with optional minor improvements)

What This Validates

Using this example, you can verify that the Code Review Agent:

- Identifies structural weaknesses
- Detects potential security risks
- Evaluates performance implications
- Assesses test coverage gaps
- Provides actionable, structured feedback



Severity Classification

The `/code_review` agent categorizes feedback as follows:

Severity Level	Meaning
Critical Issues	Must be resolved before merge
Major Concerns	Significant improvement required
Suggestions	Recommended enhancements
Minor Notes	Style or documentation refinements

This structure ensures consistent and actionable reviews.

Best Practices

To get high-quality review output:

- Provide context in the PR description.
 - Reference related design or planning documents.
 - Mention specific areas of concern if needed.
 - Ensure the PR is focused and not excessively large.
 - Include test cases where applicable.
-

Governance and Review Process

The `/code_review` agent:

- Assists reviewers
- Highlights potential risks



- Standardizes review depth

However, it does not replace human oversight. All critical decisions must be validated by maintainers or reviewers before merging.

Limitations

- The agent does not execute code.
 - It does not run test suites.
 - It evaluates based on static analysis and visible diffs.
 - It assumes repository conventions unless specified.
-

Summary

The `/code_review` agent enables structured, consistent, and thorough code evaluation directly within GitHub workflows.

It enhances review quality, reduces oversight risk, and supports engineering best practices across the organization.

